



中山大學
SUN YAT-SEN UNIVERSITY



中山大學
SUN YAT-SEN UNIVERSITY

数组

學大山中立國

中山大学计算机学院



主讲人: 陆勇



目录

CONTENTS

01

数组的概念

02

数组的定义与初始化

03

数组相关操作

04

多维数组、字符数组

05

数组与函数



数组 (Array) 的概念

向量与序列数据如何表示：

- **向量(vector)**，例如：2d向量，3d向量：
- **序列(sequence)**，高维向量，例如：
- **文字(literal)**，例如：

" Doe, a deer, a female deer\n Ray, a drop of golden sun "

这里，是向量或序列的一个**元素(element)**或**项(item)**；是下标(*index*)，表示第几个元素；是**长度**，表示向量或序列由多少个元素组成。



数组 (Array) 的概念 - 应用案例

某市气象局记录了一个月降水数据，例如：

0,3.5,0,0,0,0,0,0,0,0,0,4.2,0,0,0,0,0,0,0,0,7.0,0,0,0,0,0,0,0,-1

请计算该市一次降水的平均值？

C语言



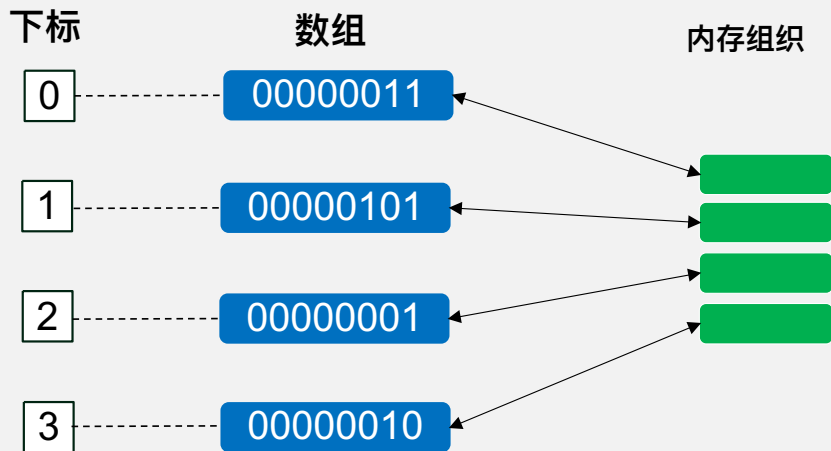


数组 (Array) 的概念 - 一种数据结构

在计算机科学中，数组数据结构 (array data structure)，简称**数组 Array**，是由**相同类型**的元素 (**element**) 的集合所组成的数据结构，分配一块**连续的内存**来存储。利用元素的**索引下标 index** 可以计算出该元素对应的存储地址。

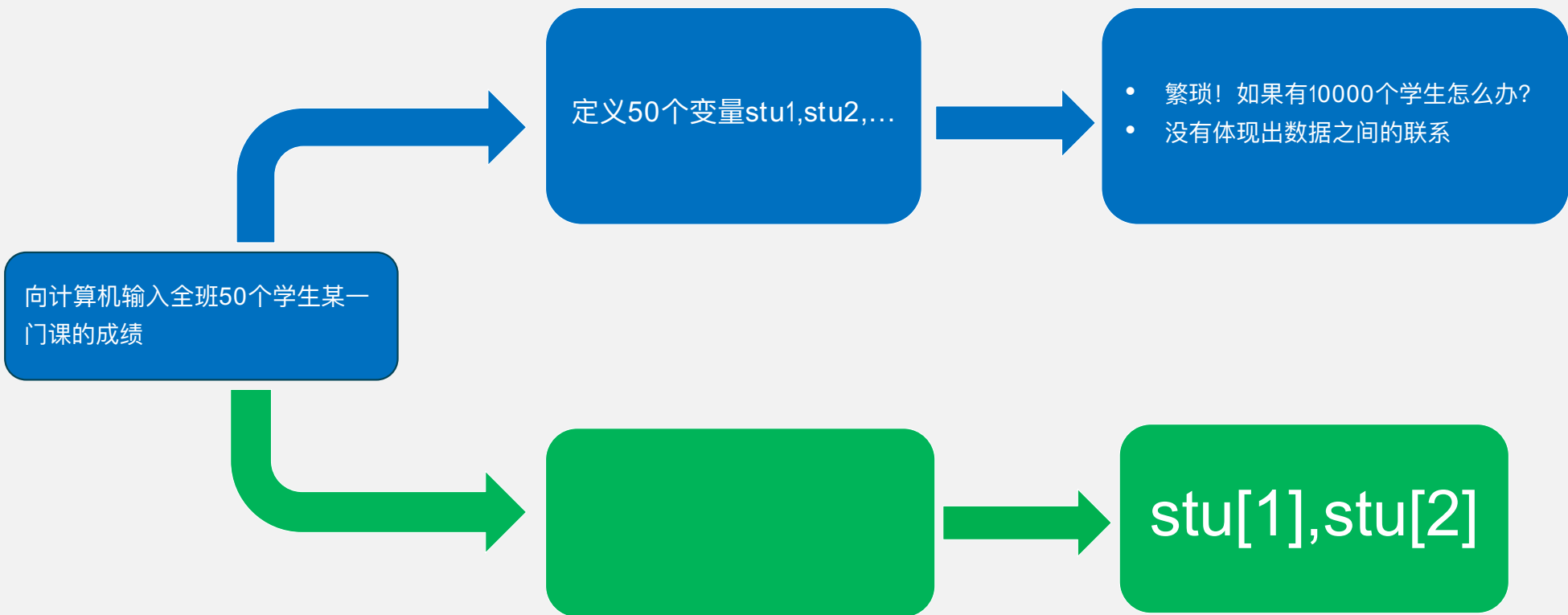
最简单的数据结构类型是一维数组。一个数组类型

描述了连续分配的非空的具有特定元素对象类型的对象集合。这些元素对象的类型称为元素类型 (element type)。**数组类型由元素类型与元素的数目确定。**





数组 (Array) 的概念 - 为什么要使用数组





一维数组的定义

类型说明符 数组名[**整数常量表达式**]

int stu[50]

stu[0]	stu[1]	stu[2]	stu[3]	stu[4]		stu[46]]	stu[47]]	stu[48]]	stu[49]]
--------	--------	--------	--------	--------	-------	--------------	--------------	--------------	--------------

- 注意!**C语言数组的下标从0开始!** 不存在数组元素stu[50]

-

在定义数组时，需要指定数组中元素的个数，方括号中的常量表达式用来表示元素的个数，即**数组长度**



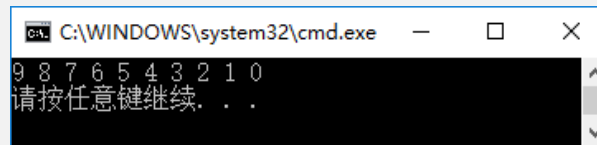
一维数组的初始化

1. 对所有元素赋值，例如 `int a[5]={0,1,2,3,4}`
2. 对部分元素赋值，例如 `int a[5]={0,1}`，此时 **剩余的元素会被自动赋0值**
3. 不指定数组长度，例如 `int a[]={0,1,2}`，但是，如果数组长度与提供初值的个数不相同，则方括号中的数组长度不能省略，
`int a[3]={0,1,2}`。

数组操作 - 使用下标访问数组元素

对10个数组元素依次赋值为0,1,2,3,4,5,6,7,8,9，要求按逆序输出。

```
#include<stdio.h>
int main()
{
    int i,a[10];
    for(i=0; i<=9;i++) // 对数组元素a[0]~a[9]赋值
        a[i]=i;
    for(i=9;i>=0;i--) // 输出a[9]~a[0]共10个数组元素
        printf("%d ",a[i]);
    printf("\n");
    return 0;
}
```



```
C:\WINDOWS\system32\cmd.exe
9 8 7 6 5 4 3 2 1 0
请按任意键继续. . .
```



第1个for循环使a[0]~a[9]的值为0~9。

第2个for循环按a[9]~a[0]的顺序输出各元素的值。

What is a array?

- ☒ A An array is a series of elements of the same type in contiguous memory locations
- ☐ B An array is a series of element
- ☐ C An array is a series of elements of the same type placed in non-contiguous memory locations
- ☐ D None of the mentioned

提交

Which of the following correctly declares an array?

- ☒ A `int array[10];`
- ☐ B `int array;`
- ☐ C `array{10};`
- ☐ D `array array[10];`

提交

What will be the output of the this program?

```
#include <stdio.h>
int main ()
{
    int array[] = {0, 2, 4, 6, 7, 5, 3};
    int n, result = 0;
    for (n = 0 ;n < 7 ;n++) {
        result += array[n];
    }
    printf("%d\n", result);
    return 0;
}
```

- ☐ A 25
- ☐ B 26
- ☒ C 27
- ☐ D None of the mentioned

提交



数组操作 - 应用案例

某市气象局记录了一个月降水数据，例如：

0,3.5,0,0,0,0,0,0,0,0,0,4.2,0,0,0,0,0,0,0,0,7.0,0,0,0,0,0,0,0,-1

请计算该市一次降水的平均值？

```
/*average rain*/
#include<stdio.h>

int main() {
    double data[100];
    int n=0;

    /*读入一个数组，以 -1 作为结束*/
    do {
        scanf("%lf",&data[n]);
        n++;
    } while (data[n-1]!=-1);
    n--;
```

```
int raindays = 0;
double rain_total = 0;
for (int i=0; i<n; i++) {
    if (data[i] > 1e-7) {
        raindays++;
        rain_total += data[i];
    }
}
if (raindays)
    printf("Raindays %d, average rainfall\n",raindays,rain_total/raindays);
else
    printf("No rain.\n");
}
```



数组常见定义错误

```
#define N 5

int main() {
    int arr1[]; // 没有声明元素数量

    int m=5;
    int arr2[m]; // C99可变长度数组/VLA (仅适用自动变量)

    int arr3[N];
    arr3 = {1, 2, 3, 4, 5}; // 数组标识符是值（地址），不能作为左值

    int arr4[N] = {1, 2, 3, 4, 5, 6}; // 初始化超长度 (warning)

    int arr5[N] = {1, 2, 3, 4, 5}, a[N];
    a = arr5; // a 不是左值

    return 0;
}
```



数组常见处理

- 需要获得数组的长度

```
#define LEN_ARRAY(a) (sizeof(a) / sizeof(a[0]))
```

- `sizeof(array)` 返回该数组定义的字节 (bytes) 数
- 类型隐式转换
 - `int a[] = {1, 2, 3.14, 4, 5};`
- 数组名是地址值（右值），它也是其第一个元素的地址，即
 - `a == &(a[0])`
 - 因此, `*a == *&(a[0]) == a[0]`，是左值
- 数组元素赋零值
 - `memset(arr, 0, sizeof(arr));`



二维数组简介

现在有一个3*3的矩阵，需要保存在计算机当中。

如果创建一个数组，应当是二维的，第一维代表第几行，第二维代表第几列。

类型说明符 数组名[常量表达式][常量表达式]

```
int mat[3][3]
```

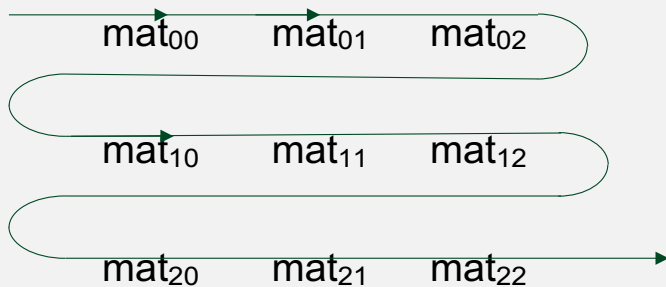
mat[0] —— mat[0][0] mat[0][1] mat[0][2]

mat[1] —— mat[1][0] mat[1][1] mat[1][2]

mat[2] —— mat[2][0] mat[2][1] mat[2][2]

二维数组存储

```
int mat[3][3]
```



- 用**矩阵形式**（如3行3列形式）表示二维数组，是**逻辑**上的概念，能形象地表示出行列关系。而在**内存**中，各元素是连续存放的，不是二维的，是**线性的**。



二维数组的初始化

(1)分行给二维数组赋初值。

```
int
```

```
mat[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

(2)可以将所有数据写在一个花括号内，按数组元素在内存中的排列顺序对各元素赋初值。

```
int
```

(3)可以对部分元素赋初值，3,4,5,6,7,8,9,10,11,12};

对于多维数组，除第一维外其他维的长度均不可省略

```
int mat[3][4]={1},{5},{9}};
```

①

```
int mat[3][4]={1},{0,6},{0,0,11}};
```

②

```
int mat[3][4]={1},{5,6}};
```

③

```
int mat[3][4]={1},{},{9}};
```

④

①

1 0 0 0

5 0 0 0

9 0 0 0

②

1 0 0 0

0 6 0 0

0 0 11 0

③

1 0 0 0

5 6 0 0

0 0 0 0

④

1 0 0 0

0 0 0 0

9 0 0 0

(4)如果对全部元素都赋初值(即提供全部初始数据)，则定义数组时对第1维的长度可以不指定，但第2维的长度不能省。

```
int
```

```
int mat[][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

在定义时也可以对部分元素赋初值而省略第1维的长度，但应分行赋初值。

```
int mat[][4]={{0,0,3},{},{0,10}};
```

What is the output of this C code? `#include <stdio.h>`

```
void main()
{
    int a[2][3] = {1, 2, 3, 4, 5};
    int i = 0, j = 0;
    for (i = 0; i < 2; i++)
        for (j = 0; j < 3; j++)
            printf("%d", a[i][j]);
}
```

☒ A

1 2 3 4 5 0

☐ B

1 2 3 4 5 junk

☐ C

1 2 3 4 5 5

☐ D

Run time error

提交

What is the output of this C code?

```
#include <stdio.h>
void f(int a[][3]) {
    a[0][1] = 3;
    int i = 0, j = 0;
    for (i = 0; i < 2; i++)
        for (j = 0; j < 3; j++)
            printf("%d", a[i][j]); }
int main(){
    int a[2][3] = {0};
    f(a); }
```

- ☒ A 0 3 0 0 0 0
- ☐ B Junk 3 junk junk junk junk
- ☐ C Compile time error
- ☐ D All junk values

提交



字符数组简介

在C语言中，是将字符串作为字符数组来处理的。

在实际工作中，人们关心的往往是字符串的有效长度而不是字符数组的长度。

为了测定字符串的实际长度，C语言规定了一个“字符串结束标志”，以字符'\0'作为结束标志。

注意

- C系统在用字符数组存储字符串常量时会自动加一个'\0'作为结束符。
- 在定义字符数组时应估计实际字符串长度，保证数组长度始终大于字符串实际长度。
- 如果在一个字符数组中先后存放多个不同长度的字符串，则应使数组长度大于最长的字符串的长度。



字符串与字符数组

```
printf("How do you do?\n");
```

在向内存中存储时，系统自动在最后一个字符'\n'的后面加了一个'\0'，作为字符串结束标志。在执行printf函数时，每输出一个字符检查一次，看下一个字符是否为'\0'，遇'\0'就停止输出。

```
char c[]={"I am happy"}; // 可省略花括号
```

```
或 char c[]="I am happy";
```

用一个字符串(注意字符串的两端是用**双引号**而不是单引号括起来的)作为字符数组的初值。

注意

- 数组c的长度不是10，而是11。因为字符串常量的最后由系统加上一个'\0'。

```
char c[]={'I', ' ', 'a','m', ' ', 'h','a','p','p','y','\0'};
```

```
char c[]={'I', ' ', 'a','m', ' ', 'h','a','p','p','y'};
```



字符数组的输入

```
char c[6];  
scanf("%s",c);
```

从键盘输入:

China

系统会自动在China后面加一个'\0'结束符。

```
char str1[5],str2[5],str3[5];  
scanf("%s%s%s",str1,str2,str3);
```

如果利用一个scanf函数输入多个字符串，则应在输入时以**空格**分隔。

从键盘输入:

How are you?

由于有空格字符分隔，作为3个字符串输入。

str 1:	H	o	w	\0	\0
str 2:	a	r	e	\0	\0
str 3:	y	e	u	?	\0

```
char str[13];  
scanf("%s",str);
```

从键盘输入:

How are you?

由于系统把空格字符作为输入的字符串之间的分隔符，因此只将空格前的字符"How"送到str中。

H	o	w	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0
---	---	---	----	----	----	----	----	----	----	----	----	----

字符数组的输入

注意

- scanf函数中的输入项如果是字符数组名，**不要再加地址符&**，因为在C语言中数组名代表该数组第一个元素的地址(或者说数组的起始地址)。



```
scanf("%s", &str); // str前面不应加&
```

- 若数组占6个字节。数组名c代表地址2000。可以用下面的输出语句得到数组第一个元素的地址。

```
printf("%o",c); // 用八进制形式输出数组c的起始地址
```

```
printf("%s",c); // 输出数组c中的各个字符，直到遇'\0'为止
```

- 实际上是这样执行的：按字符数组名c找到其数组第一个元素的地址，然后逐个输出其中的字符，直到遇'\0'为止。

c数组	
2000	C
2001	h
2002	i
2003	n
2004	a
2005	\0



字符数组的输出

(1) 逐个字符输入输出。用格式符“%c”输入或输出一个字符。

(2) 将整个字符串一次输入或输出。用“%s”

格式符，意思是对字符串(string)的输入输出。

```
#include <stdio.h>
int main()
{
    char c[]="China";
    printf("%s\n",c);
    return 0;
}
```

```
#include <stdio.h>
int main()
{
    char c[15]={' ',' ',' ','a','m',' ',' ','a',' ','s','t','u','d','e','n','t','.'};
    int i;
    for(i=0;i<15;i++)
        printf("%c", c[i]);
    printf("\n");
    return 0;
}
```

(1) 输出的字符中不包括结束符'\0'。

(2) 用“%s”格式符输出字符串时，printf函数中的输出项是字符数组名，而不是数组元素名。

(3) 如果数组长度大于字符串的实际长度，也只输出到遇'\0'结束。

(4) 如果一个字符数组中包含一个以上'\0'，则遇第一个'\0'时输出就结束。



数组与函数

如何将数组作为参数传入某个函数？

```
/*vector add*/
#include<stdio.h>
#define LEN_ARR(a) (sizeof(a) / sizeof(a[0]))
void vadd(int a[const], const int b[], const size_t sz);

int main(){
    int a[9] = {1,2,3,4,5,6,7,8,9};
    vadd(a,a,LEN_ARR(a));
    for (int i=0;i<LEN_ARR(a);i++)
        printf("%d,",a[i]);
    printf("\n");
}
void vadd(int a[const], const int b[], const size_t sz){
    for (int i=0;i<sz;i++) {
        a[i] += b[i];
//        a++; //increment of read-only parameter 'a'
//        b[i] = 1; //assignment of read-only location
    }
}
```

要点：

①

数组形参是一个变量，可保存数组的地址，用不定长数组表示，如 `b[]`。如果形参定义为 `int b[10]`，则编译忽略数组长度。

② **const** 在函数申明中应用

- `int a[const]` 表示形参 `a` 不可修改
- `const int b[]` 表示 `b` 的元素不可修改
- `const size_t sz` 表示 `sz` 不可修改

③

函数中无法获得数组的长度，通常会将数组长度作为参数传给函数，类型是 `size_t`



数组与函数

如何将多维数组作为参数传入某个函数？

```
/*matrix add*/
#include<stdio.h>

... ..
void matrix_add(int a[][3], int b[][3], size_t m);

int main(){
    int a[3][3] = {1,2,3,4,5,6,7,8,9};
    matrix_add(a,a,LEN_ARR(a));
    for (int i=0;i<LEN_ARR(a);i++) {
        for (int j=0;j<LEN_ARR(a[i]);j++)
            printf("%4d,",a[i][j]);
        printf("\n");
    }
}

void matrix_add(int a[][3], int b[][3], size_t m){
    vadd(a,b,m*LEN_ARR(a[0]));
}
```

要点：

多维数组形参仅**第一维**用不定长数组表示，如
b[][3]。

如果形参定义为 int b[10][3]，则编译解释 int
b[][3]。

多维数组形参除第一维外，长度必须是常量



数组与函数

案例程序中，三种函数声明都能处理 3x3 矩阵：

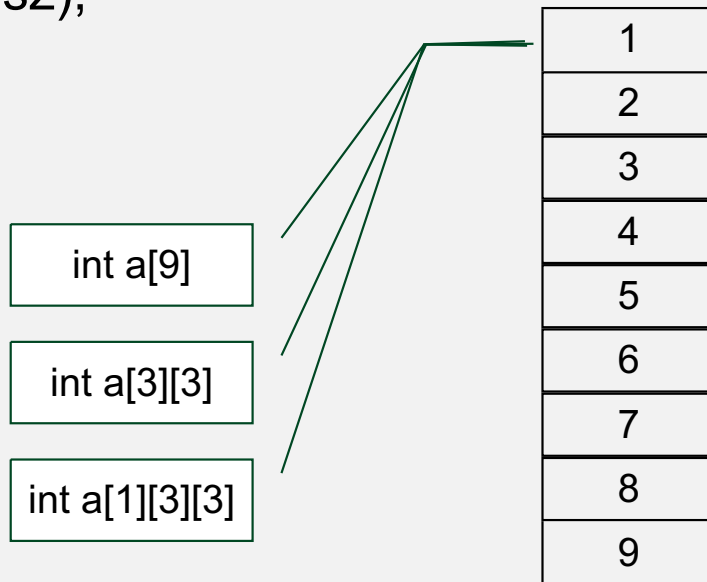
```
void vadd(int a[const], const int b[], const size_t sz);
```

```
void matrix_add(int a[][3], int b[][3], size_t sz);
```

```
void matrix_add1(int a[][3][3], int b[][3][3]);
```

你认为哪种比较最合适呢？

例如：加法，点积，外积



What will happen if in a C program you assign a value to an array element whose subscript exceeds the size of array?

- ☐ A The element will be set to 0.
- ☐ B The compiler would report an error.
- ☒ C The program may crash if some important data gets overwritten.
- ☐ D The array size would appropriately grow.

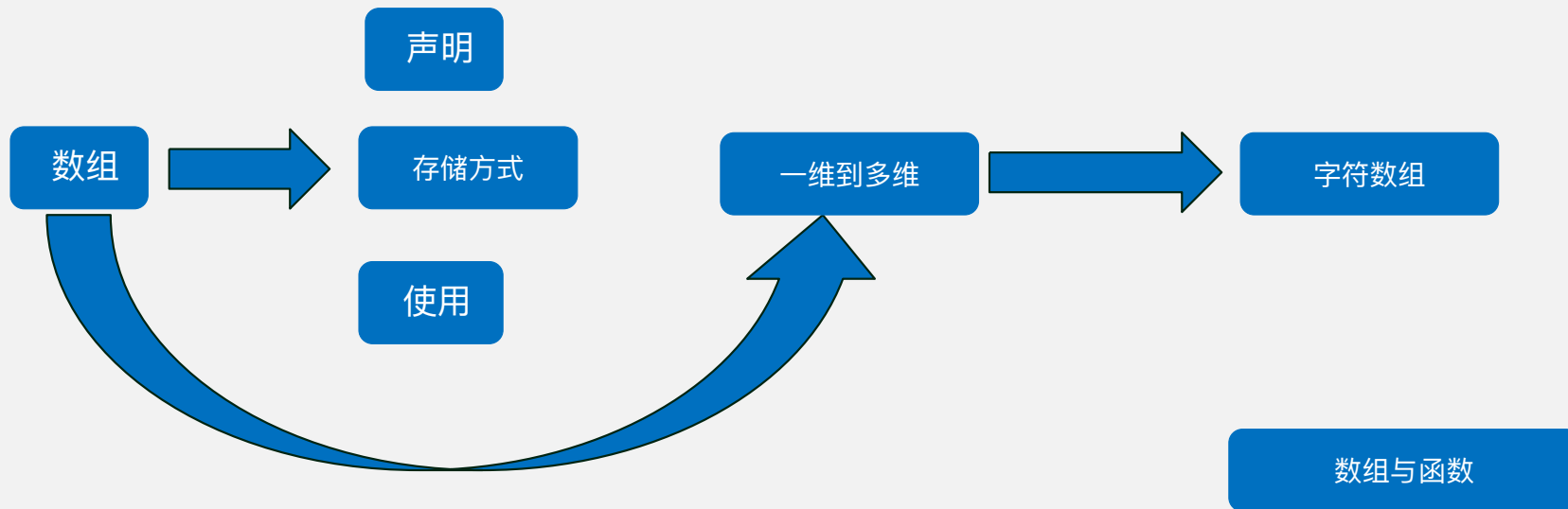
In C, if you pass an array as an argument to a function, what actually gets passed?

- ☐ A Value of elements in array
- ☐ B First element of the array
- ☒ C Base address of the array
- ☐ D Address of the last element of array

提交



总结





中山大學
SUN YAT-SEN UNIVERSITY



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

學大山中立國

中山大学计算机学院



编制人: 课题组